

Wibby — Storage Cleaning & Encryption Guide

Technical Runbook: Data Tiers, Encryption Auditing, Safe Storage Cleaning & Architecture

1. Storage Architecture Overview

Wibby implements strict 3-tier partitioning to ensure zero data leaks between users:

Storage Tier	Technology	Path / Location	Contents & Security
Database	MongoDB	mongodb://localhost:27017/wibby	User profiles, pairings, message metadata, call logs, 24h stories.
Encrypted Media	Local / OCI Storage	server/uploads/<convold>/<YYYY>/<MM>/	Partitioned voice notes, images, videos, docs with token access control.
Client Secrets	WebCrypto API	Browser localStorage (wibby_e2ee_*)	ECDH P-256 private keys; strictly never transmitted to server.

2. How to Verify Encryption (Chats, Media, Calls)

A. End-to-End Chat Encryption (ECDH P-256 + AES-GCM-256)

- **Client-side Keys:** In DevTools (F12) → Application → LocalStorage, inspect wibby_e2ee_priv_<uid>. Private keys are never uploaded to the server.
- **Safety Numbers:** In the chat UI, tap partner header → Chat Info → E2EE Security Card to cross-verify the 60-digit SHA-256 fingerprint.

B. Media Storage Protection (Token-Gated & Scoped)

- **Protected Endpoints:** Media is fetched via GET /api/conversations/:id/media/:key requiring valid Firebase Bearer token and membership.
- **Verification:** Paste any media URL in an Incognito window without logging in → instantly returns **HTTP 401 Unauthorized**.

C. Voice & Video Call Encryption (WebRTC DTLS-SRTP)

- **Transport & Ciphers:** WebRTC streams use DTLS and SRTP (AEAD_AES_128_GCM). Server acts solely as an ephemeral signaling broker and never captures media.
- **Verification:** In Google Chrome during an active call, open chrome://webrtc-internals → inspect DtlsTransport: dtlsState: connected.

3. Storage Auditing & Safe Cleaning Commands

Execute these scripts from the server/ directory to manage testing data and storage safely:

Task & Intent	Command (from server directory)	Behavior & Safety
1. Inspect Storage	<code>npm run storage:inspect</code>	Read-only. Lists all DB counts, uploads directory size, active users, and test files.
2. Verify Encryption	<code>npm run storage:verify-encryption</code>	Tests DB confidentiality, media auth gates, and cryptographic algorithms.

3. Clean Test Data Only	<code>npm run storage:clean-tests</code>	Safe cleanup: Wipes test mock audio/images and test user accounts while preserving real paired users.
4. Complete Wipe	<code>npm run storage:wipe-all</code>	Wipes all database collections and empties the uploads folder for a clean slate.

4. Clean Directory Structure Reference

The codebase is organized into modular directories to avoid bundle bloat and ensure stable execution:

- `client/src/components/`: Modular UI components (ChatHeader, MessageArea, MiniMapWidget, call/, stories/)
- `client/src/context/`: React Contexts (AuthContext, CallContext, SocketContext)
- `server/src/routes/`: Authentication, Conversations, Messages, Media, Calls, Stories
- `server/src/scripts/`: Storage auditing and verification CLI utilities
- `server/uploads/`: Scoped local media storage partitioned by conversation ID